



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления (ИУ)»

КАФЕДРА «Информационные системы и телекоммуникации (ИУЗ)»

ОТЧЕТ

по лабораторной работе №4
по курсу «Разработка Программного обеспечения»
на тему: «Разработка Flutter-приложения транспортного терминала для
считывания NFC-карт»

Студент ИУЗ-61Б
(Группа)

(Подпись, дата)

Пялкин А. А.
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Сидякин И. М.
(И. О. Фамилия)

2026 г.

1 Теоретическая часть

1.1 Цель работы

Целью лабораторной работы является разработка Flutter-приложения транспортного терминала для считывания NFC-карт с помощью модуля PN532 и библиотеки libnfc через механизм Dart FFI.

Приложение должно считывать UID транспортной карты, отображать полученные данные в интерфейсе и использовать UID карты для взаимодействия с backend-сервером, разработанным в предыдущих лабораторных работах.

1.2 Постановка задачи

В рамках лабораторной работы необходимо разработать Flutter-приложение, которое выполняет роль клиента транспортного терминала. Основным способом взаимодействия с NFC-модулем PN532 должен быть реализован через библиотеку libnfc с использованием механизма Dart FFI.

Приложение должно открывать NFC-считыватель, ожидать поднесения карты, считывать UID карты и отображать полученную информацию пользователю. После считывания UID приложение должно использовать его как номер транспортной карты.

Также необходимо реализовать связь Flutter-приложения с backend-сервером. После считывания карты приложение должно иметь возможность отправить UID карты и сумму платежа на backend для авторизации платежной транзакции. Backend проверяет существование карты, состояние блокировки, баланс и терминал, после чего возвращает результат авторизации платежа.

Дополнительно в проекте был реализован низкоуровневый диагностический способ взаимодействия с PN532 через библиотеку libserialport. В этом варианте приложение самостоятельно формирует PN532-фреймы и отправляет набор байтов в serial port. Основным способом выполнения лабораторной работы является вариант через libnfc и Dart FFI.

1.3 Используемые технологии

Для реализации клиентского приложения используется Flutter. Он позволяет создавать графические приложения с единым кодом интерфейса и

логики.

Язык Dart используется для написания логики Flutter-приложения, выполнения HTTP-запросов к backend-серверу и интеграции с нативными библиотеками.

Механизм Dart FFI используется для вызова функций нативной C-библиотеки libnfc из Dart-кода. Благодаря этому Flutter-приложение может работать с PN532 не через внешний процесс, а через прямые вызовы функций библиотеки.

Библиотека libnfc используется для взаимодействия с NFC-устройством. Она берет на себя низкоуровневую работу с PN532, открытие устройства, перевод его в режим инициатора и поиск пассивной NFC-карты.

Модуль PN532 используется как аппаратный NFC-считыватель. В работе карта считывается как пассивная цель стандарта ISO14443A, после чего из ответа извлекаются UID, ATQA и SAK.

Backend-сервер используется для авторизации платежа. Flutter-приложение отправляет на backend UID карты, серийный номер терминала и сумму операции. Ответ backend отображается в интерфейсе приложения.

1.4 Dart FFI и libnfc

Dart FFI позволяет Dart-приложению вызывать функции из динамических библиотек, написанных на C. Для этого в Dart-коде описываются типы C-функций, соответствующие им Dart-типы и выполняется поиск функций в динамической библиотеке.

В данной лабораторной работе через FFI вызываются функции libnfc: инициализация контекста, открытие NFC-устройства, перевод устройства в режим инициатора, поиск карты и закрытие устройства. Такой подход позволяет использовать PN532 из Flutter-приложения без ручного формирования команд протокола PN532.

Основная последовательность работы с libnfc выглядит следующим образом: сначала создается контекст libnfc, затем открывается устройство PN532 по connstring, после этого устройство переводится в режим инициатора. Далее приложение ожидает карту и при успешном обнаружении получает структуру с параметрами карты.

2 Практическая часть

2.1 Общая схема решения

В лабораторной работе было разработано Flutter-приложение транспортного терминала. Оно считывает NFC-карту через PN532, получает UID карты и отправляет данные на backend-сервер для авторизации платежа.

Общая схема работы системы:

```
Flutter macOS application
      |
      | Dart FFI
      v
libnfc dynamic library
      |
      v
PN532 NFC reader
      |
      v
MIFARE card UID
      |
      | HTTPS REST API
      v
Go backend server
      |
      v
SQLite database
```

Основной путь работы приложения следующий: пользователь запускает Flutter-приложение, нажимает кнопку сканирования карты, приложение через Dart FFI вызывает функции libnfc, PN532 считывает UID карты, после чего UID используется в запросе авторизации платежа к backend-серверу.

2.2 Структура проекта

Проект состоит из нескольких частей. Flutter-приложение расположено в каталоге `transport_terminal`. Пакет для работы с PN532 расположен в каталоге `pn532_cli`. Backend-сервер находится в каталоге `transport_api`.

В пакете `pn532_cli` реализованы два способа работы с PN532. Основной способ использует libnfc через Dart FFI. Дополнительный диагностический

способ использует libserialport и прямую отправку байтовых команд в serial port.

Основные файлы проекта:

```
Projects/
|-- transport_terminal/
|   |-- lib/
|   |   '-- main.dart
|   |-- pubspec.yaml
|
|-- pn532_cli/
|   |-- lib/
|   |   |-- pn532_cli.dart
|   |   '-- src/
|   |       |-- libnfc/
|   |       |   |-- libnfc_bindings.dart
|   |       |   '-- libnfc_reader.dart
|   |       |-- pn532_controller.dart
|   |       |-- pn532_device.dart
|   |       '-- pn532_frame.dart
|
'-- transport_api/
    |-- cmd/
    |-- internal/
    |-- migrations/
    |-- nginx/
    '-- docker-compose.yml
```

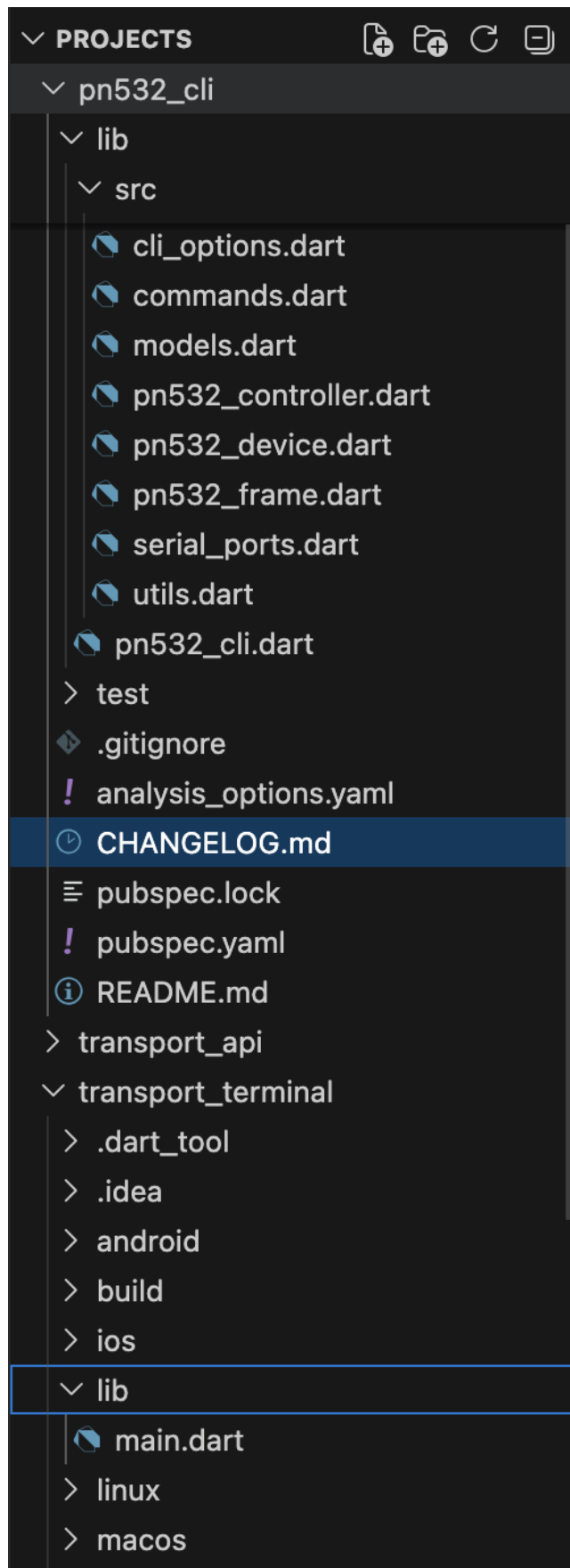


Рисунок 2.1 – Структура проекта лабораторной работы

2.3 Flutter-приложение

Flutter-приложение реализует интерфейс транспортного терминала. В интерфейсе пользователь может указать параметры подключения к PN532, выполнить считывание карты, зарегистрировать считанную карту в backend и отправить запрос авторизации платежа.

Основной экран приложения содержит блок считывания карты, блок регистрации карты и блок авторизации платежа. После считывания карты UID автоматически подставляется в поле номера карты, чтобы его можно было использовать в последующих запросах к backend.

transport_terminal

Transport Terminal

PN532 card scanner via libnfc

libnfc connstring

pn532_uart:/dev/tty.usbserial-TGJLH5CH

Example: pn532_uart:/dev/tty.usbserial-TGJLH5CH

Scan card via libnfc

No scan yet.

Register scanned card

Card UID compact

b754105e

Filled automatically after libnfc scan

Owner name

Test Passenger

Any passenger name or label

Initial balance

500

MIFARE key ID

1

Default demo key is usually 1

Blocked card

Usually disabled for new cards

Add scanned card

Card already exists.

Рисунок 2.2 – Главный экран Flutter-приложения транспортного терминала

2.4 FFI-привязки к библиотеке libnfc

Для вызова функций libnfc из Dart-кода были реализованы FFI-привязки. Они описывают сигнатуры C-функций и соответствующие им Dart-типы.

Листинг 2.1 – FFI-привязки к функциям библиотеки libnfc

```
typedef NfcInitNative = Void Function(  
    Pointer<Pointer<NfcContext>>,  
);  
  
typedef NfcOpenNative = Pointer<NfcDevice> Function(  
    Pointer<NfcContext>,  
    Pointer<Utf8>,  
);  
  
typedef NfcCloseNative = Void Function(  
    Pointer<NfcDevice>,  
);  
  
typedef NfcExitNative = Void Function(  
    Pointer<NfcContext>,  
);  
  
typedef NfcInitiatorInitNative = Int32 Function(  
    Pointer<NfcDevice>,  
);  
  
typedef NfcInitiatorSelectPassiveTargetNative = Int32 Function(  
    Pointer<NfcDevice>,  
    NfcModulation,  
    Pointer<Uint8>,  
    Int32,  
    Pointer<NfcTarget>,  
);  
  
final nfcInit = dylib.lookupFunction<  
    NfcInitNative,  
    NfcInitDart>('nfc_init');  
  
final nfcOpen = dylib.lookupFunction<  
    NfcOpenNative,  
    NfcOpenDart>('nfc_open');
```

```
final nfcInitiatorInit = dylib.lookupFunction<
    NfcInitiatorInitNative,
    NfcInitiatorInitDart>('nfc_initiator_init');

final nfcInitiatorSelectPassiveTarget = dylib.lookupFunction<
    NfcInitiatorSelectPassiveTargetNative,
    NfcInitiatorSelectPassiveTargetDart>(
    'nfc_initiator_select_passive_target',
    );
```

В листинге 2.1 показаны привязки к основным функциям libnfc. Через них Dart-код инициализирует библиотеку, открывает NFC-устройство, переводит его в режим инициатора и выполняет поиск пассивной NFC-карты.

2.5 Класс считывателя NFC-карт

Для удобной работы с libnfc был реализован класс `LibNfcReader`. Он скрывает низкоуровневые вызовы FFI и предоставляет методы открытия устройства, считывания карты и закрытия ресурсов.

Листинг 2.2 – Класс считывателя NFC-карт через libnfc

```
class LibNfcReader {
    LibNfcReader({required this.connstring});

    final String connstring;

    Pointer<NfcContext>? _context;
    Pointer<NfcDevice>? _device;

    void open() {
        final contextPtr = calloc<Pointer<NfcContext>>>();
        bindings.nfcInit(contextPtr);

        _context = contextPtr.value;

        final conn = connstring.toNativeUtf8();
        _device = bindings.nfcOpen(_context!, conn);

        if (_device == nullptr) {
            throw Exception('Cannot open NFC device');
        }
    }
}
```

```

        final result = bindings.nfcInitiatorInit(_device!);
        if (result < 0) {
            throw Exception('Cannot init NFC initiator');
        }
    }

Pn532Card scanOneCard() {
    final modulation = calloc<NfcModulation>();
    modulation.ref.nmt = NMT_ISO14443A;
    modulation.ref.nbr = NBR_106;

    final target = calloc<NfcTarget>();

    final result = bindings.nfcInitiatorSelectPassiveTarget(
        _device!,
        modulation.ref,
        nullptr,
        0,
        target,
    );

    if (result <= 0) {
        throw Exception('Card not found');
    }

    final info = target.ref.nti.nai;
    final uid = info.abtUid.asTypedList(info.szUidLen).toList();

    return Pn532Card(
        uid: uid,
        atqa: info.abtAtqa.asTypedList(2).toList(),
        sak: info.btSak,
    );
}

void close() {
    if (_device != null) {
        bindings.nfcClose(_device!);
    }
}

```

```

    if (_context != null) {
        bindings.nfcExit(_context!);
    }
}
}

```

Метод `open()` инициализирует `libnfc`, открывает PN532 по `connstring` и переводит устройство в режим инициатора. Метод `scanOneCard()` ожидает карту стандарта ISO14443A и извлекает UID, ATQA и SAK. Метод `close()` закрывает устройство и освобождает контекст `libnfc`.

Работоспособность считывания карты через PN532 и `libnfc` была проверена при демонстрации лабораторной работы. В отчёте для подтверждения реализации приведены ключевые фрагменты кода FFI-привязок, класса считывателя и интеграции считывания с Flutter-приложением.

2.6 Запуск сканирования из Flutter-приложения

Считывание карты запускается из Flutter-интерфейса. Так как ожидание карты может занимать несколько секунд, операция выполняется асинхронно. После успешного считывания UID карты записывается в поле номера карты.

Листинг 2.3 – Запуск считывания карты из Flutter-приложения

```

Future<void> scanCard() async {
    setState(() {
        isScanning = true;
        scanError = null;
    });

    try {
        final result = await runLibNfcScanInIsolate(
            connstring: connstringController.text,
            timeout: const Duration(seconds: 10),
        );

        setState(() {
            scannedCard = result.card;
            cardNumberController.text = result.card.uidCompact;
        });
    } catch (e) {
        setState(() {
            scanError = e.toString();
        });
    }
}

```

```

    });
  } finally {
    setState(() {
      isScanning = false;
    });
  }
}
}

```

В листинге 2.3 показан запуск считывания из Flutter-приложения. После успешного считывания UID карты сохраняется в состоянии страницы и может быть использован для регистрации карты или авторизации платежа.

2.7 Запуск сканирования в isolate

Операция считывания карты может блокировать выполнение программы на время ожидания карты. Чтобы интерфейс Flutter-приложения не зависал, сканирование было вынесено в отдельный isolate.

Листинг 2.4 – Запуск libnfc-сканирования в отдельном isolate

```

Future<LibNfcScanResult> runLibNfcScanInIsolate({
  required String connstring,
  required Duration timeout,
}) async {
  final receivePort = ReceivePort();

  await Isolate.spawn(
    libNfcScanIsolateEntry,
    LibNfcScanRequest(
      sendPort: receivePort.sendPort,
      connstring: connstring,
    ),
  );

  final message = await receivePort.first.timeout(timeout);

  if (message is LibNfcScanResult) {
    return message;
  }

  throw Exception('Invalid isolate response');
}

```

В листинге 2.4 показан общий принцип запуска сканирования в отдельном isolate. Основной UI-поток отправляет параметры подключения, а isolate открывает PN532 через libnfc, ожидает карту и возвращает результат обратно в интерфейс.

2.8 Регистрация карты в backend

После считывания UID карта может быть зарегистрирована в backend-сервере. Для этого Flutter-приложение отправляет POST-запрос на маршрут работы с картами. В запрос передаются номер карты, владелец, баланс, флаг блокировки и идентификатор ключа.

Листинг 2.5 – Регистрация считанной карты в backend

```
Future<void> createCard() async {
  final response = await http.post(
    Uri.parse('$apiBaseUrl/cards'),
    headers: {
      'Content-Type': 'application/json',
      'Authorization': 'Bearer $jwtToken',
    },
    body: jsonEncode({
      'card_number': cardNumberController.text,
      'owner_name': ownerController.text,
      'balance': int.parse(balanceController.text),
      'is_blocked': false,
      'key_id': int.parse(keyIdController.text),
    }),
  );

  final data = jsonDecode(response.body);

  setState(() {
    cardRegisterResult = data;
  });
}
```

Регистрация карты нужна для того, чтобы backend мог найти карту по UID при последующей авторизации платежа. Если карта уже есть в базе данных, приложение может сразу использовать её номер для выполнения платежной операции.

2.9 Авторизация платежа через backend

Главная функция терминального приложения — отправка платежной транзакции на backend. Пользователь указывает сумму и серийный номер терминала, а номер карты берется из результата считывания NFC-карты.

Листинг 2.6 – Отправка запроса авторизации платежа на backend

```
Future<void> authorizePayment() async {
  final response = await http.post(
    Uri.parse('$apiBaseUrl/terminal/transactions/authorize'),
    headers: {
      'Content-Type': 'application/json',
      'Authorization': 'Bearer $jwtToken',
    },
    body: jsonEncode({
      'terminal_serial': terminalSerialController.text,
      'card_number': cardNumberController.text,
      'amount': int.parse(amountController.text),
    }),
  );

  final data = jsonDecode(response.body);

  setState(() {
    paymentResult = data;
  });
}
```

В листинге 2.6 показана отправка запроса авторизации платежа. Flutter-приложение передает backend-серверу серийный номер терминала, UID считанной карты и сумму операции. Backend проверяет карту, терминал, блокировку и баланс, после чего возвращает результат авторизации платежа.

Результат успешной авторизации платежа представлен на рисунке 2.3.

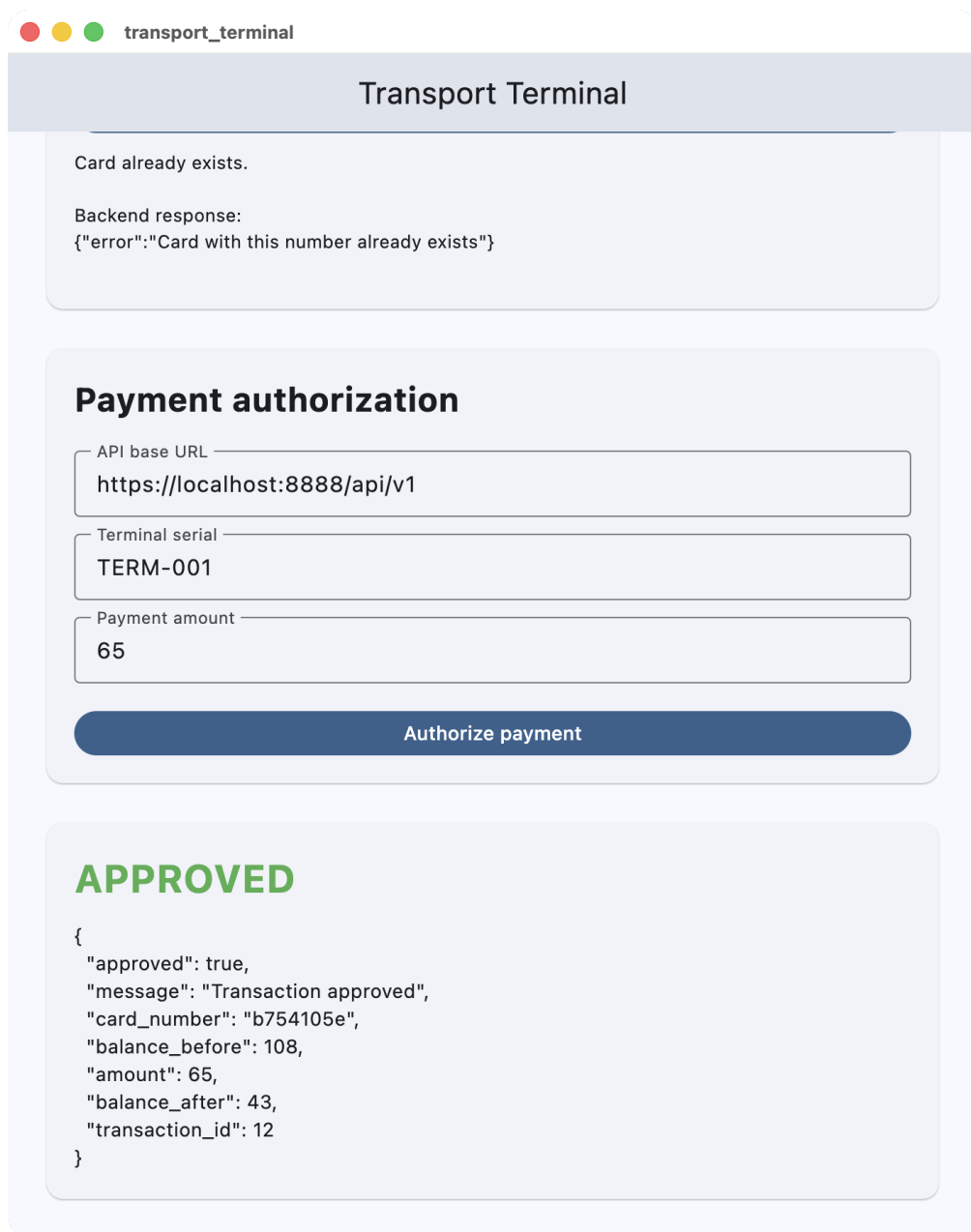


Рисунок 2.3 – Успешная авторизация платежа в Flutter-приложении

После успешной авторизации платежа backend создает запись в таблице транзакций. Проверка истории транзакций представлена на рисунке 2.4.

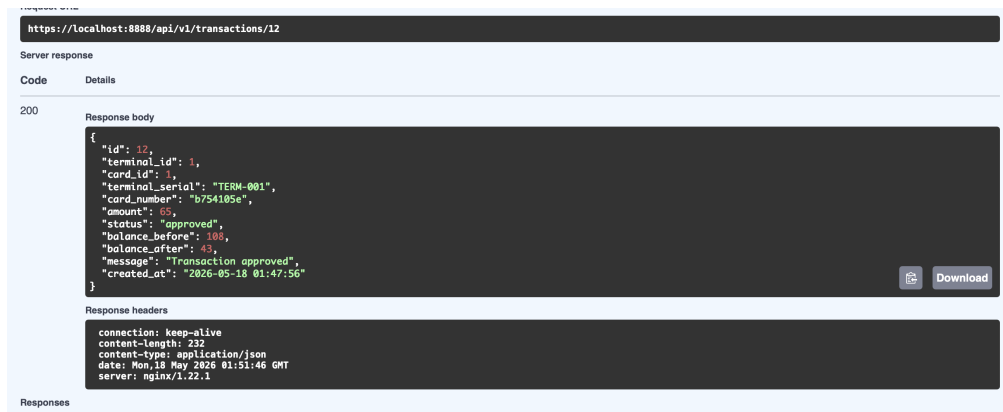


Рисунок 2.4 – Запись о выполненной транзакции на backend-сервере

2.10 Дополнительный способ через libserialport

Помимо основного способа через libnfc и Dart FFI, в проекте был реализован диагностический вариант работы с PN532 через libserialport. В этом варианте приложение работает с serial port напрямую и самостоятельно формирует PN532-фреймы.

Такой способ полезен для отладки, потому что позволяет отправлять конкретные команды PN532 и анализировать ответы устройства. Однако он требует ручной реализации протокола обмена: формирования преамбулы, длины, контрольных сумм, кода команды и разбора ответа.

Листинг 2.7 – Формирование PN532-фрейма для отправки через serial port

```

UInt8List buildPn532Frame(List<int> data) {
    final length = data.length;
    final lcs = (-length) & 0xFF;
    final dcs = (-data.fold<int>(0, (sum, byte) => sum + byte)) &
        0xFF;

    return UInt8List.fromList([
        0x00,
        0x00,
        0xFF,
        length,
        lcs,
        ...data,
        dcs,
        0x00,
    ]);
}

```

В листинге 2.7 показан принцип формирования PN532-фрейма. Этот вариант не является основным для лабораторной работы, но был реализован как дополнительная проверка и средство диагностики работы PN532.

2.11 Вывод

В ходе выполнения лабораторной работы было разработано Flutter-приложение транспортного терминала для считывания NFC-карт через PN532. Основной способ взаимодействия с модулем реализован через библиотеку `libnfc` и механизм Dart FFI.

Были реализованы FFI-привязки к `libnfc`, класс считывателя NFC-карт, запуск сканирования из Flutter-приложения, регистрация карты в `backend` и авторизация платежной транзакции. Также был реализован дополнительный диагностический способ работы с PN532 через `libserialport` и прямое формирование PN532-фреймов.

Разработанное приложение связывает аппаратную часть транспортного терминала с `backend`-сервером: UID карты используется для выполнения платежной операции, а результат авторизации отображается в интерфейсе Flutter-приложения.

3 Список литературы

1. Документация Flutter. Режим доступа: <https://docs.flutter.dev/>
2. Документация Dart. Режим доступа: <https://dart.dev/guides>
3. Документация Dart FFI. Режим доступа: <https://dart.dev/interop/c-interop>
4. Документация libnfc. Режим доступа: <https://github.com/nfc-tools/libnfc>
5. Документация PN532 User Manual. Режим доступа: <https://www.nxp.com/docs/en/user-guide/141520.pdf>
6. Документация пакета libserialport для Dart. Режим доступа: <https://pub.dev/packages/libserialport>
7. Документация пакета http для Dart. Режим доступа: <https://pub.dev/packages/http>